MIT 8103 Advanced Database Systems

SALIHU_HAUWAU_30923000

MIT 8103 Advanced Database Systems: Portfolio Summary

Case study: Haske Hotel Reservation System, Minna (83 rooms)

Student: Hauwau Salihu · **Student ID:** 301923000

Repository: https://github.com/hauwausalihu/advanced-database-portfolio-Salihu_Hauwau_301923000

Case study

Haske hotel reservation system is a simulated 83-room hotel located in Minna Niger State. It does reservations, staff book rooms, check and track availability, take payments and records check-ins and check-outs. The manager and accountants review bookings and take payments. Guest book over the phone or online. The database stores guests, rooms and room types, bookings, payments, and actual check-in and check-out times. All data is simulated. I used PostgreSQL 18 for Portfolios 1 to 3 and MongoDB 9.0.8 for Portfolios 4 and 5. Each portfolio loaded its own data set, and a later set replaced the earlier one.

Portfolio 1: Database Design and Modelling

Six tables were designed: guest, room_type, room, booking, booking_room and payment. Booking_room links a booking to its rooms, because one booking can hold several rooms and one room is booked many times. The ER diagram using draw.io, Normalization was explained from one untidy sheet through 1NF,2NF and 3NF and built the tale one at a time in PostgreSQL with primary keys, foreign keys, UNIQUE, NOT NULL and CHECK rules.

Verified result: the database refused invalid data. A room with a room type that does not exist failed the foreign key, and a room with the status Broken failed the CHECK rule. This means the rules are enforced by the database, not left to the people entering data.

Portfolio 2: Query Processing and Optimisation

Simulated data were loaded ((20,000 guests, 50,000 bookings, 50,000 booking rooms and 30,000 payments), which replaced the Portfolio 1 sample data. I added five indexes (four where a plan showed a full table scan, and one to test Q5) and ran the queries again. Times are the Execution Time reported by EXPLAIN ANALYZE, in milliseconds.

Query	Before (ms)	After (ms)	Results
Q1 guest by name	3.933	0.163	about 24 times faster
Q2 bookings in one week	7.747	0.503	about 15 times faster
Q3 one guest's bookings	12.108	0.328	about 37 times faster

Q4 revenue per month	58.976	65.196	no real change (no index added)
Q5 rooms free for a date range	42.644	24.025	about 1.8 times faster

Q1 to Q3 improved because the plans changed from reading whole tables to using indexes, and PostgreSQL does not index foreign keys automatically. Q4 needs almost every row, so I added no index, yet its time moved from 58.976 to 65.196 ms. This shows that single runs vary by several milliseconds. Each figure is one measurement on one laptop with a small data set, so the ratios are approximate. Q5 returned no free rooms because the simulated bookings overlap, so only its timing is meaningful.

Portfolio 3: Transactions and Concurrency

The test here used a small clean data set (4 guests, 5 rooms and one booking) replacing the bulk data in portfolio 2. A COMMIT saved a booking, its room and payment all together. A ROLLBACK chosen or forced by a failed insert, left no trace of the booking, so the step fails or succeeds as one. I used two SQL Shell windows to cause a doble booking, two receptionists (window A and B) saw room 202 as free for the same night and booked it both, because a key or CHECK cannot compare one booking with another. A row lock (SELECT FOR UPDATE) was done which made the second receptionist wait and see if the room was taken. I also compared READ COMMITED, where a value changed inside one of the transactions, with repeatable READ. This means that a rules between bookings need locks and transactions not tables alone.

Portfolio 4: NoSQL and Advanced Data Models

The hotel was modelled in MongoDB as three collections: guests,rooms(with room_type embedded) and bookings (with its rooms and payments embedded). A one document is a whole booking, so no booking room is needed. 4 bookings were loaded and different queries were ran (including a search inside the embedded rooms list), two aggregations and two updates. I compared a guestid lookup before index (a full collection scan) and after it (an index scan). With only four documents the time difference is too small to measure, so the change in plan is the evidence. Schema validation rejected invalid bookings, but a booking for a guest not existing was accepted because MongoDB cannot check another collection. This means documents suit reading a booking as a whole, while rules between records need extra logic.

Portfolio 5: Distributed and Cloud Database Exercise

I ran a three-node MongoDB replica set called hotelrs on my laptop, using three mongod programs on ports 27018, 27019 and 27020, each with its own data folder. One node is the primary and takes all writes, and the other two are secondaries that copy it. I loaded the hotel data into the primary and checked that a secondary returned the same bookings, and that a write sent to a secondary is refused.

A majority of three nodes is two, so one node can fail and the hotel can still take bookings. A write concern of 3 gives the strongest consistency but cannot finish when a node is down, so a majority write is safer for a hotel. Not performed: a failover test (it did not complete on my machine, so availability is explained but not demonstrated), a sharded cluster (a design only, with hotelr in the shard key for bookings, because one 83-room hotel does not need sharding) and a cloud deployment. I chose a local replica set because the free MongoDB Atlas tier has a fixed three-node setup, cannot be sharded and does not allow failover tests. Three nodes on one laptop show how replication works, but they do not protect against the laptop failing.

Individual reflection

The building of this database for haske hotel gave me an understanding on how reservation systems work. Since it is a reservation system for a hotel, the design and six tables were picked leaving out staffs and housekeeping to ensure that it doesn't become too ambiguous and go out of my scope. One of the many difficulties faced is when I try to push from git bash to my github and it doesn't appear on the github online. I always had to troubleshoot and finally understood how to do it and was able to always remember the commands to make it appear and work without issues.

References

MongoDB. Atlas Free Cluster Limitations. MongoDB Atlas documentation. https://mongodb.com/docs/atlas/reference/free-shared-limitations/

MongoDB. Install MongoDB Community on Windows. MongoDB Manual. https://mongodb.com/docs/v8.0/tutorial/install-mongodb-on-windows-unattended

Software used: PostgreSQL 18 with psql, MongoDB Community Server 9.0.8 with mongosh, draw.io, Git and GitHub. No external datasets, libraries or tutorial code were used. All data and scripts were written for this portfolio, with the AI help declared below.

AI-use declaration

Tools: Claude (Anthropic).

Help received: explaining database concepts (normalisation, execution plans, indexing, transactions, isolation levels, the document model and replication); suggesting the table design, sample data, queries, tests and MongoDB commands; guiding me through setting up Git, GitHub, PostgreSQL and MongoDB; and drafting the wording of my READMEs and this summary.

My verification: I ran the commands in psql and mongosh myself, and the results quoted here are my own output. I corrected errors as they appeared, including a script comment that was missing its two dashes and an index that had not been created.